

ОПЕРАЦИОННЫЕ СИСТЕМЫ

ЛАБОРАТОРНЫЙ ПРАКТИКУМ

ЛАБОРАТОРНАЯ РАБОТА № 5. Поток

Цель работы:

Получение практических навыков по использованию Win32 API для исследования потоков

Методические указания к выполнению лабораторной работы

Дескрипторы и идентификаторы потоков Функция `CreateProcess` возвращает идентификатор и дескриптор первого потока, выполняющегося во вновь созданном процессе. Функция `CreateThread` тоже возвращает идентификатор потока, но область действия которого – вся система. Поток может использовать функцию `GetCurrentThreadId`, чтобы получить собственный ID. Функция `GetWindowThreadProcessId` возвращает идентификатор того потока, который создал конкретное окно. Наконец, поток может вызывать функцию `GetCurrentThread` для получения собственного псевдодескриптора. Как и в случае псевдодескрипторов процессов, псевдодескриптор потока может использоваться только для вызова процесса и не может наследоваться. Можно использовать функцию `DuplicateHandle` для получения настоящего дескриптора потока по его псевдодескриптору так же, как это делается в процессах. Приоритет потоков Термин многозадачность или мультипрограммирование, обозначает возможность управлять несколькими процессами (или несколькими потоками) на базе одного процессора. Многопроцессорной обработкой называется управление некоторым числом процессов или потоков на нескольких процессорах. Компьютер может одновременно выполнять команды двух разных процессов только в многопроцессорной среде. Однако даже на одном процессоре с помощью переключения задач можно создать впечатление, что одновременно выполняются команды нескольких процессов. Синхронизация потоков Выполняющимся потокам часто необходимо каким-то образом взаимодействовать. Например, если несколько потоков пытаются получить доступ к некоторым глобальным данным, то каждому потоку нужно предохранять данные от изменения другим потоком. Иногда одному потоку нужно получить информацию о том, когда другой поток завершит выполнение задачи. Такое взаимодействие обязательно между потоками как одного, так и разных процессов. Синхронизация потоков (thread synchronization) – это обобщенный термин, относящийся к процессу взаимодействия и взаимосвязи потоков. Синхронизация потоков требует привлечения в качестве посредника самой операционной системы. Потоки не могут взаимодействовать друг с другом без ее участия. В Win32 существует несколько методов синхронизации потоков. В зависимости от конкретной ситуаций один метод более предпочтителен, чем другой. Критические секции Один из методов синхронизации потоков состоит в использовании критических секций (critical sections). Это единственный метод синхронизации потоков, который не требует привлечения ядра Windows. (Критическая секция не является объектом ядра). Однако этот метод может использоваться только для синхронизации потоков одного процесса. Критическая секция – это некоторый участок кода, который в каждый момент времени может выполняться только одним из потоков. Если код, используемый для инициализации массива, поместить в критическую секцию, то другие потоки не смогут войти в этот участок кода до тех пор, пока первый поток не завершит его выполнение. До использования критической секции необходимо инициализировать ее с помощью процедуры Win32 API `InitializeCriticalSection()`. После заполнения записи в программе можно создать критическую секцию, поместив некоторый участок ее текста между вызовами функций `EnterCriticalSection()` и

LeaveCriticalSection(). Синхронизация с использованием объектов ядра Многие объекты ядра, включая процесс, поток, файл, мьютекс, семафор, уведомление об изменении файла и событие, могут находиться в одном из двух состояний – "свободно" (signaled) и "занято" (nonsignaled). Посредством вызова функций WaitForSingleObject и WaitForMultipleObjects поток приостанавливает свое выполнение до того момента, когда заданный объект (или объекты) перейдет в состояние "свободно". Мьютекс – это объект ядра, который можно использовать для синхронизации потоков из разных процессов, создается с помощью вызова функции CreateMutex. Он может принадлежать или не принадлежать некоторому потоку. Если мьютекс принадлежит потоку, то он находится в состоянии "занято". Если данный объект не относится ни к одному потоку, то он находится в состоянии "свободно". Другими словами, принадлежать для него означает быть в состоянии "занято". События используются в качестве сигналов о завершении какой-либо операции. Однако в отличие от мьютексов они не принадлежат ни одному потоку. Например, поток А создает событие с помощью функции CreateEvent и устанавливает объект в состояние "занято". Поток В получает дескриптор этого объекта, вызвав функцию OpenEvent, затем вызывает функцию WaitForSingleObject, чтобы приостановить работу до того момента, когда поток А завершит конкретную задачу и освободит указанный объект. Когда это произойдет, система выведет из состояния ожидания поток В, который теперь владеет информацией, что поток А завершил выполнение своей задачи. Семафоры. Существует еще один метод синхронизации потоков, в котором используются семафорные объекты API. В семафорах применен принцип действия мьютексов, но с добавлением одной существенной детали. В них заложена возможность подсчета ресурсов, что позволяет заранее определенному числу потоков одновременно войти в синхронизируемый участок кода. Для создания семафора используется функция CreateSemaphore(). Семафоры могут быть полезны при совместном использовании ограниченных ресурсов. Предположим, имеется три приложения, каждое из которых должно выполнить вывод на печать, а у компьютера только два параллельных порта. Установив семафор с начальным значением счетчика ресурсов, равным двум, можно заставить приложения запрашивать сервис печати только тогда, когда есть свободный параллельный порт. Ждущий таймер (waitable timer) представляет собой новый тип объектов синхронизации, поддерживаемый в Windows NT версии 4.0 и выше. Это полноценный объект синхронизации, который может использоваться для организации задержки в одном или нескольких приложениях. Ждущий таймер работает в трех режимах. В режиме "ручного сброса" таймер переходит в установленное состояние при истечении заданной задержки и остается установленным до тех пор, пока функция SetWaitableTimer не задаст новую задержку. В режиме "автоматического сброса" таймер переходит в установленное состояние при истечении заданной задержки и остается установленным до первого успешного вызова функции ожидания. В этом режиме он напоминает объект Event в режиме автоматического сброса, поскольку каждый раз при истечении времени задержки разрешается выполнение лишь одной нити. Наконец, ждущий таймер может выполнять функции интервального таймера, который перезапускается с заданной задержкой после каждого срабатывания объекта.

Ход выполнения лабораторной работы

КОД ПРОГРАММЫ:

```
#include <iostream>
#include <list>

class LinkedList {
private:
    std::list<int> list;
```

```

public:
    void addElement(int value) {
        list.push_back(value);
    }

    void printList() {
        std::cout << "List: ";
        for (auto it = list.begin(); it != list.end(); ++it) {
            std::cout << *it << " ";
        }
        std::cout << std::endl;
    }
};

class Client {
private:
    LinkedList& list;

public:
    Client(LinkedList& l) : list(l) {}

    void addToLinkedList(int value) {
        list.addElement(value);
    }
};

class LinkedListMutex {
public:
    void run() {
        LinkedList linkedList;

        const int numClients = 5;

        for (int i = 0; i < numClients; ++i) {
            Client client(linkedList);
            client.addToLinkedList(i + 1);
        }

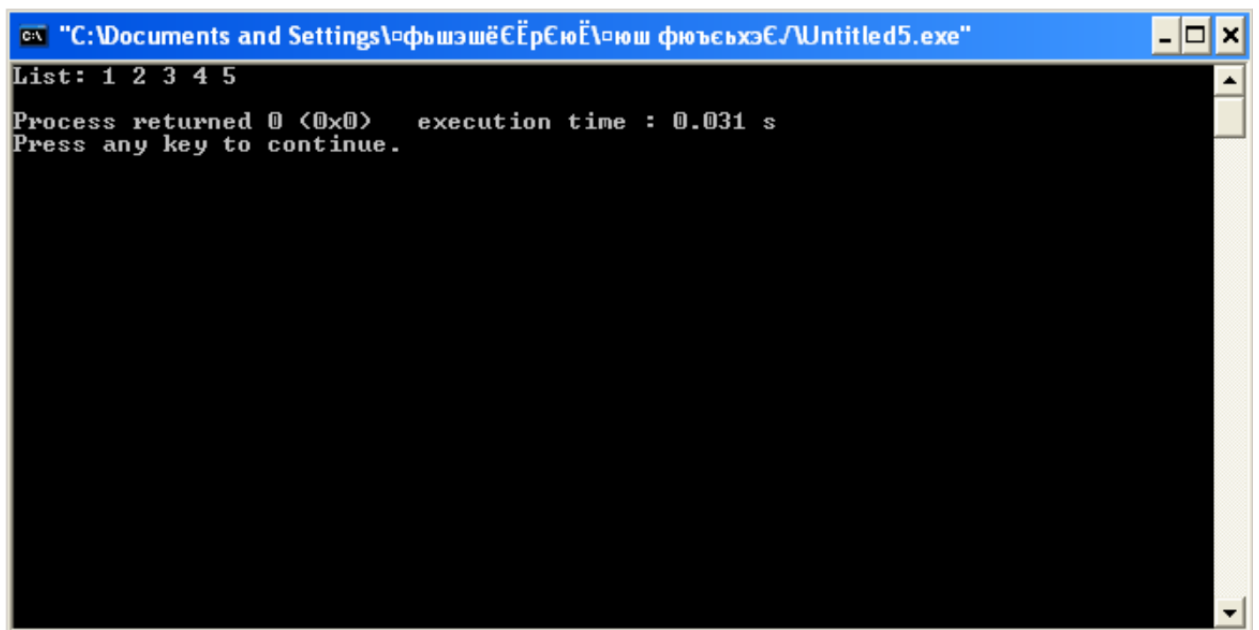
        linkedList.printList();
    }
};

int main() {
    LinkedListMutex program;
    program.run();

    return 0;
}

```

РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ:



```
C:\ "C:\Documents and Settings\фьюсьхэс\Untitled5.exe"
List: 1 2 3 4 5
Process returned 0 (0x0)   execution time : 0.031 s
Press any key to continue.
```